

/elite_options_system_v2_4/

|

|— config_v2_4.json # Main configuration (API keys, paths, regime rules, metric params)

|— run_system_dashboard.py # Primary script to launch the interactive Dash dashboard

|— README_v2_4.md # This guide

|— requirements.txt # Lists all Python package dependencies

|— .env # (Optional) For storing environment variables like API keys

|

|— data_management/ # Handles all data fetching, initial processing, and storage

| |— __init__.py # Makes 'data_management' a Python package

| |— fetcher.py # Fetches raw data (e.g., from ConvexValue API)

| |— initial_processor.py # Cleans raw data, basic transformations, saves to 'raw_data_cache'

| |— historical_data_manager.py # Manages short-term history of key underlying/metric data

| |— raw_data_cache/ # Stores fetched raw data (e.g., daily API pulls)

| |— processed_data_cache/ # Stores data after initial_processor (ready for core analytics)

|

|— core_analytics_engine/ # The "brain" of the system

| |— __init__.py # Makes 'core_analytics_engine' a Python package

| |— its_orchestrator.py # Main class (e.g., IntegratedTradingSystemV2_4) that orchestrates analytics

| |— metrics_calculator.py # Calculates all granular & aggregate metrics (V2.3 refined + new V2.4)

| |— market_regime_engine.py # Classifies the current market regime

- | |— signal_generator.py # Generates discrete trading signals based on metrics & regime
- | |— recommendation_logic.py # Formulates strategy recommendations, manages active ones
- | |— trade_parameter_optimizer.py # Calculates optimal/dynamic targets and stops
- | |— utils_engine.py # Utility functions specific to the analytics engine
- |
- |— dashboard_application/ # Dash application for visualization and interaction
 - |— __init__.py # Makes 'dashboard_application' a Python package
 - |— app_main.py # Main Dash app definition and server instantiation
 - |— layout_manager.py # Dynamically builds dashboard layout based on selected "mode"
 - |— callback_manager.py # Registers and organizes all Dash callbacks
 - |— styling.py # CSS variables, Plotly templates
 - |— assets/ # For static assets like custom.css, images
 - |— custom.css
 - |
 - |— modes/ # Sub-package for different dashboard views/modes
 - |— __init__.py
 - |— main_dashboard_display.py # Logic for the core/main dashboard layout and its specific charts
 - |— volatility_mode_display.py # Logic for the "Volatility Deep Dive" mode
 - |— flow_mode_display.py # Logic for the "Flow Breakdown" mode
 - |— structure_mode_display.py # Logic for "GEX/DEX Structure" (SDAGs, DAG) & MSPI
 - |— time_decay_mode_display.py # Logic for "Time Decay & Pinning" mode

II. Component Classification and Uses

1. Root Directory (/elite_options_system_v2_4/):

- **Use:** Contains global configuration, the main application runner, documentation, and environment setup files.
- **Key Files:**
 - config_v2_4.json: Central hub for all settings – API keys, file paths, metric calculation parameters, Market Regime Engine rules, signal thresholds, recommendation logic parameters. *Critical for user customization.*
 - run_system_dashboard.py: The single script the user runs to start the entire system. It initializes necessary components (like the its_orchestrator) and launches the Dash web application.

2. data_management/ Package:

- **Classification:** Data Ingestion, Initial Processing, and Persistence.
- **Use:** Responsible for acquiring raw data from external sources (like ConvexValue), performing initial cleaning and validation, and storing it in a structured way for the analytics engine. Manages short-term historical data needed for certain calculations (e.g., average IV, ATR, historical metric distributions for dynamic thresholds).
- **Modules:**
 - fetcher.py: Makes API calls, handles API rate limits, retries.
 - initial_processor.py: Converts raw API responses into standardized Pandas DataFrames, handles basic data type issues, saves to a local cache (e.g., daily files in raw_data_cache/ or processed_data_cache/).
 - historical_data_manager.py: Maintains rolling windows of essential daily data (e.g., underlying IV, OHLC for ATR) loaded from processed_data_cache/ or an external source if the API doesn't provide sufficient history.

3. core_analytics_engine/ Package:

- **Classification:** Core Logic, Metric Calculation, Regime Analysis, Signal Generation, Recommendation Formulation, and State Management.
- **Use:** This is the analytical heart. It takes processed data, calculates all derived metrics, determines the market regime, generates trading signals, formulates actionable strategy recommendations (with dynamic parameters), and manages the lifecycle of these recommendations.
- **Modules:**
 - `its_orchestrator.py`: Contains the main `IntegratedTradingSystemV2_4` class. This class is instantiated by `run_system_dashboard.py`. Its methods are called by the dashboard callbacks to get fresh analysis. It coordinates calls to all other modules within this package. Manages active_recommendations.
 - `metrics_calculator.py`: Contains functions/classes to calculate *all* the detailed metrics (V2.3 metrics refined with accurate flow data, plus new V2.4 metrics like `vri_0dte`, `GIB`, `HP_EOD`, `NVP`, rolling net flows, etc.). Takes data from `data_management/` outputs.
 - `market_regime_engine.py`: Defines and classifies the current market regime based on a wide array of inputs from `metrics_calculator.py`.
 - `signal_generator.py`: Generates discrete trading signals (e.g., "MSPI Support Break," "Vanna Cascade Alert") based on metric values and the current market regime.
 - `recommendation_logic.py`: Takes signals and regime information to formulate categorized recommendations (Directional, Volatility, Range, Cautionary). Includes logic for dynamic conviction scoring.
 - `trade_parameter_optimizer.py`: Calculates initial and dynamically adjusted stop-loss and target levels for recommendations, factoring in regime, ATR, and S/R levels.

4. `dashboard_application/` Package:

- **Classification:** User Interface (UI) and User Experience (UX).
- **Use:** Provides the interactive web-based dashboard for visualizing all the insights generated by the `core_analytics_engine`. Allows users to switch between different "modes" to focus on specific analytical areas.

- **Modules:**

- `app_main.py`: Defines the Dash app object, sets up the server, and typically calls a function from `layout_manager.py` to build the initial layout.
- `layout_manager.py`: Contains functions to generate the overall structure of the dashboard and the layouts for different "modes." It will decide which charts/tables are displayed based on the active mode.
- `callback_manager.py`: Contains all Dash callbacks. These callbacks respond to user interactions (e.g., changing modes, selecting symbols, date ranges), trigger recalculations in the `core_analytics_engine` (via the `its_orchestrator` instance), and update the charts and tables in the current mode's layout.
- `modes/`: Each file within this sub-package (e.g., `main_dashboard_display.py`, `volatility_mode_display.py`) would ideally contain:
 - A function to generate the specific layout for that mode's charts/tables.
 - Functions that create the Plotly figure objects for the charts relevant to that mode (these functions would be called by the `callback_manager.py`).

III. Anaconda Environment Setup

(This section would be identical to your V2.3 guide, listing Python version and necessary packages. Ensure requirements.txt is up-to-date with any new libraries V2.4 might introduce, though the core ones like pandas, numpy, dash, plotly, convexlib remain.)

1. Open Anaconda Prompt

2. Create environment (if not already done):

```
conda create -n options_env_v2_4 python=3.11
```

3. Activate environment:

```
conda activate options_env_v2_4
```

4. Install packages from requirements.txt:

```
pip install -r requirements.txt
```

(Ensure convexlib is in rIV. System Startup & Workflow

1. Configuration (config_v2_4.json):

- **Crucial First Step:** User updates API keys, data storage paths (raw_data_cache, processed_data_cache), and any initial parameters for the Market Regime Engine, metric calculations, or signal thresholds.

2. Initial Data Fetch & Process (Can be run manually or scheduled):

- Navigate to /elite_options_system_v2_4/ in Anaconda Prompt.
- `python -m data_management.fetcher` (Fetches data, saves to raw_data_cache)
- `python -m data_management.initial_processor` (Cleans raw data, saves to processed_data_cache)
- `python -m data_management.historical_data_manager` (If separate script to prime historical stores for ATR/IV averages)

3. Launch the Dashboard Application:

- Navigate to /elite_options_system_v2_4/ in Anaconda Prompt.
- Run the main application runner:

requirements.txt or installed separately as per V2.3 guide)

```
python run_system_dashboard.py --config-path config_v2_4.json
```

1.

- (Include other arguments like --port, --production as needed).
- run_system_dashboard.py will:
 - Load config_v2_4.json.
 - Instantiate the main IntegratedTradingSystemV2_4 from core_analytics_engine.its_orchestrator. This instance will be passed to the dashboard callbacks.
 - Initialize the Dash app from dashboard_application.app_main.
 - Start the web server.

2. Using the Dashboard:

- Open a web browser to the provided address (e.g., <http://127.0.0.1:8050/>).
- The dashboard will load, typically defaulting to the "Main Dashboard Mode."
- Callbacks triggered by timers or user interaction will:
 - Instruct the IntegratedTradingSystemV2_4 instance to fetch the latest processed data (from data_management caches if needed, though often the processor would have run recently).
 - Trigger the its_orchestrator to run its full analytical pipeline: metrics_calculator -> market_regime_engine -> signal_generator -> recommendation_logic (including trade_parameter_optimizer) -> update_active_recommendations_and_manage_state.
 - The results (recommendations, chart data for the current mode) are passed back to the dashboard for display.
- User can switch between different "modes" to view specialized sets of charts and analyses.

graph TD

A[User Interaction via Web Browser] -->|Requests Data/Mode Change| B[dashboard_application/app_main.py + callback_manager.py];

B -->|Triggers Analysis via Runner| C[run_system_dashboard.py];

C -->|Calls Methods On| D[core_analytics_engine/its_orchestrator.py Instance];

E[data_management/ (fetcher, initial_processor, historical_manager)] -->|Provides Processed Data & History| D;

D -->|Uses| F[core_analytics_engine/metrics_calculator.py];

F -->|Outputs Rich Metrics DF| G[core_analytics_engine/market_regime_engine.py];

G -->|Outputs Current Regime| H[core_analytics_engine/signal_generator.py];

H -->|Outputs Discrete Signals| I[core_analytics_engine/recommendation_logic.py];

I -- Uses --> J[core_analytics_engine/trade_parameter_optimizer.py];

I -->|Manages/Outputs Recommendations| D;

D -->|Returns Data to Runner/Callbacks| C;

C -->|Updates| B;

B -->|Updates Layout via layout_manager.py & mode_*.py files| A;

K[config_v2_4.json] -. -> |Loaded by| C;

K -. -> |Referenced by| D;

K -. -> |Referenced by| E;

K -. -> |Referenced by| F;

K -. -> |Referenced by| G;

K -. -> |Referenced by| H;

K -. -> |Referenced by| I;

K -. -> |Referenced by| J;